

Dossier Technique de Validation

Plateforme d'Automatisation Intelligente des Tests Applicatifs

Architecture, Commandes, Techniques de Test — Préparation Soutenance

Auteur :	Ilyes Chaabani	Date :	23 Juin 2026
Projet :	PFE — Automatisation Intelligente des Tests	Type :	Dossier technique de défense

Ce dossier rassemble l'ensemble des éléments techniques nécessaires à la validation du projet : l'architecture microservices (Spring Boot / Next.js), la stack technologique et ses versions, les commandes de build et d'exécution, le modèle de données PostgreSQL, le pipeline de génération de tests assistée par IA (LLM local Ollama), le moteur d'exécution Maven/TestNG, le moteur de tests fonctionnels web à trois oracles, les tests Android via Appium, la génération de rapports PDF, et une section de questions/réponses anticipées pour la soutenance.

4	5	LLM	Postgre SQL
Microservices	Couches de test	Génération IA	Base unique

Table des Matières

1.	Présentation & Objectifs	
2.	Architecture Microservices	
3.	Stack Technique & Versions	
4.	Modèle de Données & Hiérarchie de Configuration	
5.	Authentification & Sécurité (JWT, GitHub OAuth)	
6.	ms_gestion — Gestion & Génération de Tests	
7.	Génération de Tests par IA (M1 / ScenarioBuilder)	
8.	ms-execution — Moteur d'Exécution	
9.	Techniques de Test Couvertes (Pyramide)	
10.	Moteur de Tests Fonctionnels Web (3 Oracles)	
11.	Tests Android (Appium W3C)	
12.	Rapports PDF & KPIs	
13.	Frontend Next.js	
14.	Commandes — Build, Run, Test	
15.	Conventions & Pièges Techniques	
16.	Questions / Réponses de Soutenance	

1. Présentation & Objectifs

La plateforme automatise la création et l'exécution de tests applicatifs (unitaires, d'intégration, fonctionnels web et mobile) en s'appuyant sur un **LLM local** (deepseek-coder:6.7b via Ollama) pour générer du code de test ciblé et analyser les échecs. Elle couvre le cycle complet : connexion du dépôt GitHub, navigation du code source, génération guidée de cas de test, exécution réelle via Maven, et restitution sous forme de rapports PDF.

1.1 Problématique

- Écrire et maintenir des tests est coûteux en temps et exige une expertise technique élevée.
- Les tests générés naïvement par IA compilent rarement (mauvais packages, mocks manquants, données irréalistes).
- Les outils existants ne relient pas génération, exécution réelle, et analyse des échecs dans un flux unique.

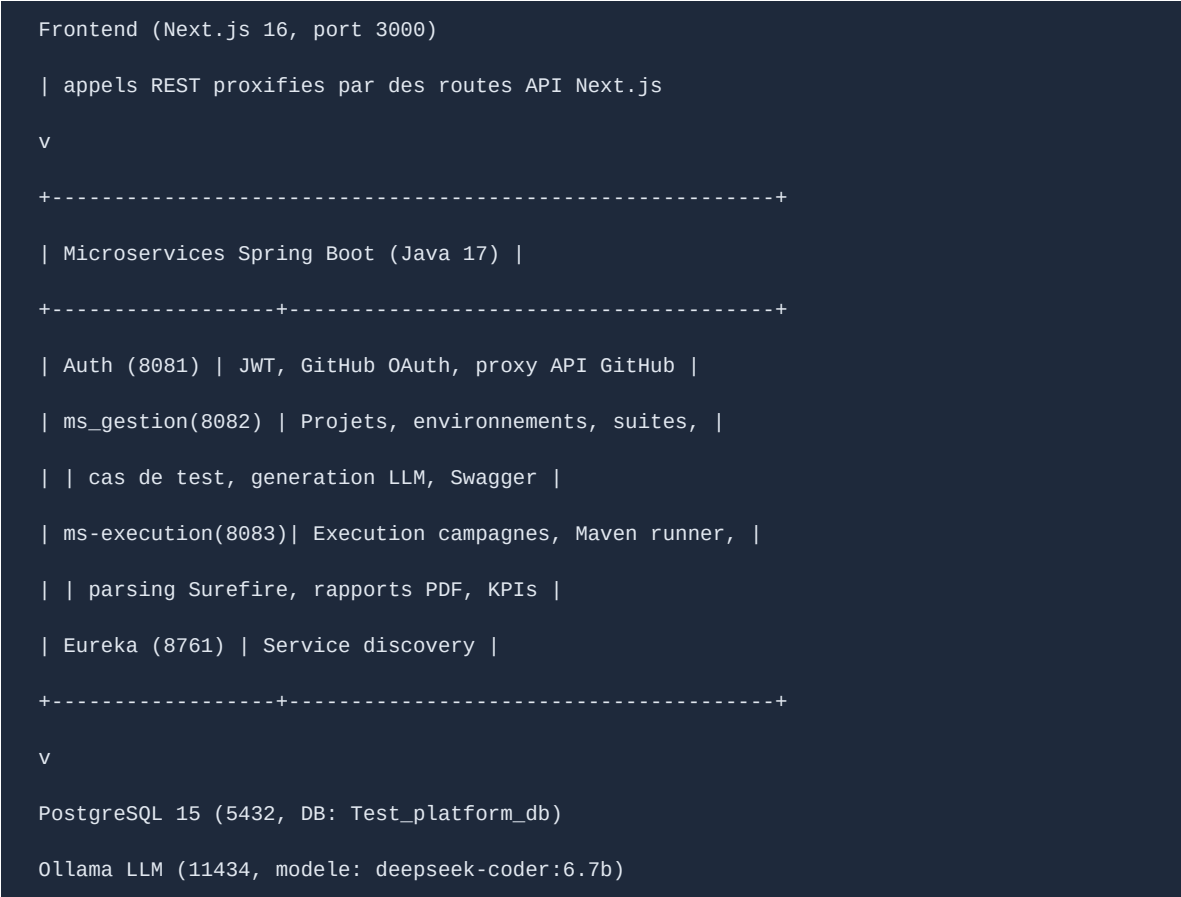
1.2 Réponse apportée

- **Génération structurée** : prompts contraints par méthode + scénario + schéma Swagger (pas de prompt libre).
- **Exécution réelle** : clone du dépôt, injection du test, lancement Maven/TestNG, parsing Surefire.
- **Boucle qualité** : analyse IA des échecs avec classification du type d'erreur.
- **Multi-couches** : unitaire, intégration, fonctionnel web (3 oracles), et mobile Android (Appium).

Argument de défense : La valeur n'est pas « l'IA écrit des tests » mais « l'IA écrit des tests qui compilent et s'exécutent réellement », grâce à un prompt structuré et à une boucle d'exécution/analyse fermée.

2. Architecture Microservices

Architecture distribuée : un frontend Next.js communique via des routes proxy avec quatre microservices Spring Boot enregistrés dans Eureka, partageant une base PostgreSQL unique, et un serveur LLM Ollama local.



2.1 Rôle de chaque service

Service	Port	Responsabilités
Eureka	8761	Annuaire de découverte de services (démarré en premier)
Auth	8081	Émission/validation JWT, OAuth GitHub, proxy de l'API GitHub (WebClient)
ms_gestion	8082	CRUD projets/environnements/suites/cas, génération LLM, données Swagger, campagnes
ms-execution	8083	Clone Git, exécution Maven, parsing Surefire, analyse IA des échecs, rapports PDF

2.2 Décision d'architecture — base partagée

Les quatre services partagent une seule base PostgreSQL. **ms_gestion** est propriétaire du schéma (ddl-auto=update, il crée les colonnes). **ms-execution** lit des miroirs en lecture seule de ces tables (ddl-auto=validate) : il refuse de démarrer si une colonne attendue manque — d'où des migrations SQL manuelles côté exécution.

Pourquoi une base partagée ? Choix pragmatique pour un PFE : évite la duplication de données et la synchronisation inter-services. Le découplage est assuré au niveau applicatif (entités miroir read-only), pas au niveau base.

3. Stack Technique & Versions

Couche	Technologie	Version / Détail
Frontend	Next.js (App Router) + React + TypeScript	Next.js 16, Turbopack, port 3000
Frontend tests	Vitest	npm run test
Backend	Spring Boot, Java	Java 17
Découverte	Spring Cloud Netflix Eureka	port 8761
Persistance	Spring Data JPA + Hibernate	PostgreSQL 15
Base de données	PostgreSQL	DB Test_platform_db, port 5432
Documentation API	springdoc-openapi (Swagger)	/v3/api-docs sur ms_gestion
LLM	Ollama — deepseek-coder	modèle 6.7b, port 11434, GPU 4GB
Framework de test généré	TestNG + Mockito	TestNG 7.9, Mockito 5.7
Tests intégration	@SpringBootTest + H2 / Testcontainers	AbstractTestNGSpringContextTests
Test web fonctionnel	Selenium WebDriver (Chrome)	JavaScript injecté, sans CDP
Test mobile	Appium (API W3C REST)	UiAutomator2, JDK HttpClient
Build	Maven (mvnw wrapper)	Surefire configuré TestNG
Infra locale	Docker Compose	PostgreSQL + PgAdmin
Rapports	iTextPDF	rapports d'exécution PDF
Auth	JWT + GitHub OAuth	cookies access_token / refresh_token

4. Modèle de Données & Hiérarchie de Configuration

Principe central : **toute la configuration technique vit sur l'Environnement**, pas sur le Projet ni la Suite. Le projet n'est qu'un nom + description.

```
Project -> name + description seulement

Environment -> baseUrlWeb, baseUrlApi, gitRepoUrl, gitBranch, databaseType

TestSuite -> name + type (UNIT|INTEGRATION|WEB) + modulePath (optionnel)

TestCase -> title + code genere (IA) ou scriptPath (manuel)

Campaign -> project + environment + cas de test selectionnes -> Run
```

4.1 Entités principales

Entité	Champs clés	Notes
Project	name, description	Volontairement minimal
Environment	baseUrlWeb, baseUrlApi, gitRepoUrl, gitBranch, databaseType	Porte toute la config technique
TestSuite	name, type, modulePath	type = UNIT INTEGRATION WEB
TestCase	title, code/scriptPath, active, flaky, maxDurationSeconds, springProfile, testData, targetClassName	active=false ignoré, flaky=true rejoué
Campaign	project, environment, testCasesIds, runMode	runMode = ALL SELECTED

4.2 Particularités PostgreSQL / JPA

- **test_cases.test_data** est de type JSONB : l'entité ms-execution doit utiliser `@JdbcTypeCode(SqlTypes.JSON) + columnDefinition="JSONB"`, pas TEXT.
- **environments** a git_repo_url, git_branch, database_type (l'ancienne colonne JSONB variables est dépréciée).
- **ms_gestion** auto-crée les colonnes (ddl-auto=update) ; **ms-execution** exige des ALTER TABLE manuels (ddl-auto=validate).
- Entités ms_gestion : Lombok ; entités miroir ms-execution : getters/setters manuels (read-only).

5. Authentification & Sécurité

5.1 JWT par cookies

L'authentification repose sur des JWT stockés en cookies (`access_token`, `refresh_token`). Le frontend appelle les API avec `credentials: 'include'` via `apiFetch()`. Aucun store global (Redux/Zustand).

5.2 Intégration GitHub

- Connexion via **GitHub OAuth** côté service Auth.
- Le service Auth expose un **proxy de l'API GitHub** (`GitHubClient` en `WebClient`) — y compris la liste des fichiers Java d'un repo : `/api/github/repos/{owner}/{repo}/java-files`.
- `WebClient` configuré avec `InsecureTrustManagerFactory` en dev (SSL).
- Clone Git authentifié par PAT embarqué dans l'URL :
`https://oauth2:TOKEN@github.com/...`
- Propriété `github.token` présente dans `application.properties` de `ms-execution` et `ms_gestion`.

Endpoints internes : `ms_gestion` expose des endpoints `/api/internal/` SANS contrôle `SecurityUtils`, pour les appels service-à-service (ex. génération automatique du rapport en fin de campagne, sans contexte utilisateur).

6. ms_gestion — Gestion & Génération de Tests

6.1 Responsabilités

- CRUD des projets, environnements, suites et cas de test.
- Génération de code de test via le LLM (`LlmService`) et de données de test (`TestDataGeneratorService`).
- Exposition des schémas Swagger/OpenAPI pour guider la génération.
- Gestion des campagnes et de leurs cas de test (ajout/retrait).

6.2 Endpoints LLM clés

Méthode	Endpoint	Rôle
POST	<code>/api/llm/generate-test</code>	Génère un test à partir d'un payload structuré + skeleton + testData
POST	<code>/api/llm/generate-testdata</code>	Génère des données de test JSON réalistes selon le scénario
GET	<code>/api/llm/swagger?schema=Name</code>	Renvoie les définitions de champs d'un schéma (name, type, enum, required)

6.3 Gestion des cas de test d'une campagne

Méthode	Endpoint	Action
GET	.../campaigns/{id}/available-testcases	Cas du projet non encore dans la campagne
POST	.../campaigns/{id}/testcases	Ajoute des cas (bloqué si RUNNING)
DELETE	.../campaigns/{id}/testcases/{tid}	Retire un cas (bloqué si RUNNING)

7. Génération de Tests par IA (M1 / ScenarioBuilder)

Le cœur de l'innovation : transformer un prompt libre fragile en un **prompt structuré** qui force le LLM à produire un test précis, compilable et exécutable.

7.1 Flux du wizard (UNIT / INTEGRATION)

- 1 Le wizard parcourt les fichiers source via l'API GitHub (sans cloner).
- 2 L'utilisateur sélectionne une classe -> le frontend extrait un skeleton (~15 lignes, capé à 1500 caractères).
- 3 ScenarioBuilder étape 1 : choisir la méthode (dropdown des méthodes publiques parsées).
- 4 Étape 2 : choisir le scénario (HAPPY_PATH / EXCEPTION / NULL_INPUT / WRONG_INPUT / BOUNDARY).
- 5 Étape 3 : le LLM génère un JSON de données de test selon le schéma REQUEST Swagger -> l'utilisateur valide/édite.
- 6 Étape 4 : les assertions sont auto-construites depuis le schéma RESPONSE Swagger + les données validées.
- 7 Le payload structuré (methodName, scenarioType, expectedBehavior) + skeleton + testData part au LLM.

7.2 Règles imposées au LLM

Catégorie	Règle
Package	UNIT -> package suites.unit; / INTEGRATION -> package suites.integration;
Framework	TestNG + Mockito (UNIT, sans contexte Spring) ; @SpringBootTest + H2/Testcontainers (INTEGRATION)
Nommage	Méthode forcée : test_{methodName}_{scenarioType}() — exactement UN @Test par scénario
Mocks	@InjectMocks sans new() ; @BeforeMethod avec MockitoAnnotations.openMocks(this)
Statique	Si SecurityUtils.getCurrentUserId() utilisé -> MockedStatic + teardown @AfterMethod
Repositories	Mocker TOUS les repositories utilisés (findById, save, existsBy...)
Données	Utiliser les valeurs de testData dans les setters de la requête

7.3 Génération des données par scénario

Scénario	Stratégie de données
HAPPY_PATH	Valeurs valides et réalistes

NULL_INPUT	Un champ requis mis à null
WRONG_INPUT	Valeur hors enum / hors contrainte
BOUNDARY	Valeurs limites (min/max, longueurs)
EXCEPTION	Contexte déclenchant une exception métier

7.4 Nettoyage de la sortie LLM

La sortie est post-traitée : suppression des blocs Markdown (``java, ``) et des tokens spéciaux deepseek (<|begin_of_sentence|>, <|end_of_sentence|>). Le LLM tourne sur GPU 4GB VRAM, d'où le cap de 1500 caractères sur le skeleton.

8. ms-execution — Moteur d'Exécution

8.1 Pipeline d'exécution d'une campagne

- 1 `runCampaign(campaignId, selectedTestCasesIds) clone env.gitRepoUrl @ env.gitBranch (PAT en URL).`
- 2 Pour UNIT/INTEGRATION : écrit le code généré dans `src/test/java/suites/unit/` (ou `/integration/`).
- 3 Injecte les dépendances dans le `pom.xml` (Mockito 5.7, TestNG 7.9, H2, Testcontainers...).
- 4 Lance : `mvnw test -Dtest=suites.unit.ClassName -Dsurefire.provider=testng`.
- 5 Parse `target/surefire-reports/TEST-*.xml` pour les résultats par méthode.
- 6 Appelle le LLM pour analyser les échecs (classification du type d'erreur).
- 7 Stocke les résultats dans `execution_results`, puis génère le rapport PDF.

8.2 Fonctionnalités d'exécution

Fonction	Comportement
<code>active=false</code>	Le cas de test est ignoré
<code>flaky=true</code>	Le test échoué est rejoué une fois
<code>maxDurationSeconds</code>	Timeout Maven par test (fallback global : 30 min)
<code>springProfile</code>	Injecté en <code>-Dspring.profiles.active={value}</code>
<code>testData</code>	Écrit dans un fichier temp, passé en <code>-Dtest.data.file=path</code>
<code>databaseType</code>	Résolu d'abord depuis le cas de test, puis l'environnement
<code>runMode SELECTED</code>	Filtre l'exécution aux <code>testCasesIds</code> spécifiés

8.3 Analyse IA des échecs (LlmAnalysisService)

- **extractRelevantErrors()** : extrait uniquement les lignes [ERROR], cannot find symbol, AssertionError, NullPointerException (pas les N premiers caractères arbitraires).
- **detectErrorType()** : classe en COMPILATION_ERROR, ASSERTION_FAILURE, NULL_POINTER, TIMEOUT, RUNTIME_EXCEPTION.
- Le prompt force une réponse structurée : **Cause / Correction / Conseil**.

9. Techniques de Test Couvertes (Pyramide)

La plateforme couvre plusieurs niveaux de la pyramide de test, chacun avec son framework et ses contraintes de génération propres.

Niveau	Framework / Outil	Caractéristiques
Unitaire	TestNG + Mockito	Sans contexte Spring, mocks de tous les repositories, MockedStatic si besoin
Intégration	@SpringBootTest + H2 / Testcontainers	AbstractTestNGSpringContextTests, vraie couche persistance
Fonctionnel Web	Selenium + JS injecté (3 oracles)	Détection de formulaires, 4 scénarios déterministes
Mobile (APK)	Appium W3C REST (UiAutomator2)	Emulateur Android Docker, arbre d'accessibilité
UX / Exploratoire	Agent IA + screenshots	Navigation, évaluation UX, HITL en mode supervisé

9.1 Les 4 scénarios de test fonctionnel

Scénario	Précondition	Objectif
EMPTY_SUBMIT	≥ 1 champ required	Vérifier le rejet d'une soumission vide
INVALID_FORMAT	type email/url/tel ou pattern	Vérifier le rejet d'un format invalide
BOUNDARY	minLength/maxLength/min/max	Vérifier le rejet hors bornes
HAPPY_PATH	toujours (dernier)	Vérifier l'acceptation de données valides

10. Moteur de Tests Fonctionnels Web (3 Oracles)

Pour décider si un formulaire valide correctement ses entrées, le moteur combine trois sources d'information indépendantes. Aucune seule ne suffit ; leur fusion couvre la quasi-totalité des applications web modernes (HTML5 natif, frameworks JS, soumissions AJAX).

Oracle	Mécanisme	Force / Limite
1 — HTML5 Validity	Lecture de input.validity (valueMissing, typeMismatch, patternMismatch, tooLong/Short, range...)	Déterministe et natif / inopérant sur composants React/Vue custom
2 — DOM Errors	Scan de sélecteurs (.error, [aria-invalid], .is-invalid, .Mui-error, [role=alert]...)	Couvre les validations custom / dépend des conventions CSS
3 — Réseau XHR/fetch	Monkey-patch de XMLHttpRequest + wrap de fetch, capture des codes HTTP	Résout les formulaires AJAX silencieux / pas de verdict sur erreur réseau (code 0)

10.1 Table de décision (fusion)

HTML5	DOM	Réseau	Verdict
invalid	—	—	PASS — validation native active
valid	présentes	—	NEEDS_REVIEW — règle métier ?
valid	absentes	2xx	PASS — formulaire AJAX accepté
valid	absentes	4xx	FAIL HIGH — serveur rejette données valides
valid	absentes	5xx	FAIL CRITICAL — bug serveur
valid	absentes	aucun	WARN — soumission silencieuse

10.2 HITL — Human-in-the-Loop

En mode **SUPERVISED**, un verdict NEEDS_REVIEW (confiance < 0.6) suspend le test et interroge le testeur via WebSocket STOMP. `waitForInput(id, 5 min)` bloque jusqu'à réponse (OK -> PASS, BUG -> FAIL, IGNORE -> WARN) ou timeout (verdict inchangé). Le champ `human_validated` est persisté. En mode AUTO, les verdicts incertains sont gardés tels quels.

11. Tests Android (Appium W3C)

AppiumDriverService pilote un émulateur Android conteneurisé en utilisant uniquement `java.net.http.HttpClient` (JDK 11+) contre l'API W3C WebDriver — **aucune dépendance java-client** (qui entrerait en conflit avec le BOM Selenium de Spring Boot).

Méthode	Rôle
<code>ensureEmulatorRunning()</code>	Démarre le conteneur Docker (<code>--privileged --device /dev/kvm, -e APPIUM=true</code>)
<code>waitForDeviceBoot(180)</code>	Poll <code>adb shell getprop sys.boot_completed</code> jusqu'à '1'
<code>waitForAppiumReady(300)</code>	Poll GET <code>/status</code> jusqu'à <code>ready=true</code> (port 4725 -> 4723 conteneur)
<code>createSession(apkPath)</code>	POST <code>/session</code> (UiAutomator2), capture <code>screenWidth/Height</code>
<code>getInteractiveElements()</code>	Parse l'arbre d'accessibilité XML, extrait les bounds exacts
<code>tap / swipe / typeText</code>	W3C Actions API avec coordonnées scalées

Point clé de défense : L'usage de l'arbre d'accessibilité (`TAP_ELEMENT` par index) évite l'imprécision des coordonnées estimées par l'IA dans un espace d'écran différent du réel (540px vs 1440px).

12. Rapports PDF & KPIs

Chaque campagne génère automatiquement un rapport PDF (iTextPDF) en fin d'exécution.

Section du rapport	Contenu
Page de couverture	Nom projet, version, environnement, badge de statut
Résumé exécutif	KPIs (total/succès/échecs/erreurs/durée) + barre de progression
Contexte	Projet, suites, cas, environnement (repo, branche, type DB)
Résultats détaillés	Carte par test avec type d'erreur + 1re ligne d'erreur significative
Analyse IA	Lignes d'erreur extraites + analyse structurée Cause/Correction/Conseil
Métriques	Top 5 tests les plus lents, durée moyenne par type (bar chart)
Tests flaky	Tests marqués instables
Recommandations	Analyse IA complète par test échoué (non tronquée)
Annexe	Scripts IA générés

Données du rapport : Les rapports utilisent les endpoints `/api/internal/` de `ms_gestion` (sans `SecurityUtils`) pour résoudre les vrais noms (projet/campagne) lors d'une génération automatique sans contexte utilisateur.

13. Frontend Next.js

Aspect	Détail
Routing	App Router (Next.js 16), pages sous Frontend/app/
Proxy API	Routes API Next.js relaient vers Auth (8081), ms_gestion (8082), ms-execution (8083)
Auth fetch	apiFetch() avec credentials:'include' contre NEXT_PUBLIC_API_URL
État	JWT par cookies, pas de Redux/Zustand
Types	Frontend/types/ms-gestion.ts pour tous les types domaine
Services	Frontend/services/ (axiosClient) ; Frontend/lib/api-client.ts (fetch)

13.1 Composants clés

- **SourceClassWizard** : navigateur de classes en 3 étapes via l'API GitHub.
- **ScenarioBuilder** : wizard M1 (méthode -> scénario -> données -> assertions).
- **EvaluationLiveView** : vue temps réel des verdicts (couleurs dynamiques, thumbnails, boîte HITL).
- **CampaignCard, FormDialog, ConfirmDialog** : composants transverses.

13.2 Intégration Swagger

ms_gestion expose /v3/api-docs (springdoc). Le proxy frontend /api/llm/swagger?schema=TestCaseResponse renvoie les définitions de champs (name, type, enum, required), utilisées par ScenarioBuilder pour auto-remplir les checklists d'assertions et générer les données de test.

14. Commandes — Build, Run, Test

14.1 Infrastructure

```
cd infrastructure && docker-compose up -d # PostgreSQL + PgAdmin
```

14.2 Backend (ordre de démarrage)

```
cd Backend/EurekaServeur && mvn spring-boot:run # 1. d'abord  
cd Backend/AuthenticationMicroservice && mvn spring-boot:run # 2.  
cd Backend/ms_gestion && mvn spring-boot:run # 3.  
cd Backend/ms-execution && mvn spring-boot:run # 4. apres migration SQL
```

14.3 Frontend

```
cd Frontend && npm install && npm run dev # Turbopack, port 3000
```

14.4 Tests

```
cd Frontend && npm run test # Vitest  
cd Backend/ms_gestion && mvn test # JUnit/TestNG  
cd Backend/ms-execution && mvn test
```

14.5 Commande Maven exécutée par le moteur

```
mvnw test -Dtest=suites.unit.ClassName -Dsurefire.provider=testng \  
-Dspring.profiles.active={profile} \  
-Dtest.data.file={tempPath}
```

14.6 Ordre de démarrage complet

Ordre	Service	Port	Commande
1	PostgreSQL	5432	docker-compose up -d (infrastructure/)
2	Eureka	8761	mvn spring-boot:run (EurekaServeur/)
3	Auth	8081	mvn spring-boot:run (AuthenticationMicroservice/)
4	ms_gestion	8082	mvn spring-boot:run (ms_gestion/)
5	ms-execution	8083	mvn spring-boot:run (ms-execution/)

6	Frontend	3000	npm run dev (Frontend/)
—	Ollama LLM	11434	ollama run deepseek-coder:6.7b

15. Conventions & Pièges Techniques

Sujet	Convention / Piège
ddl-auto	ms_gestion=update (crée colonnes) ; ms-execution=validate (ALTER TABLE manuel obligatoire)
JSONB	test_cases.test_data : @JdbcTypeCode(SqlTypes.JSON) + columnDefinition JSONB, jamais TEXT
Framework de test	TOUJOURS TestNG (jamais JUnit) — Surefire est configuré pour TestNG
Lombok	ms_gestion utilise Lombok ; entités miroir ms-execution = getters/setters manuels
Packages générés	UNIT -> suites.unit ; INTEGRATION -> suites.integration (sinon écriture fichier KO)
Auth clone Git	PAT embarqué : https://oauth2:{token}@github.com/... (ms-execution & ms_gestion)
github.token	Présent dans application.properties des deux services
Endpoints internes	/api/internal/ de ms_gestion sans SecurityUtils pour appels service-à-service

Avant tout redémarrage de ms-execution : Comme ddl-auto=validate, exécuter les migrations SQL (ALTER TABLE / CREATE TABLE) sur Test_platform_db, sinon le service refuse de démarrer.

16. Questions / Réponses de Soutenance

Q. Pourquoi un LLM local plutôt qu'une API cloud (OpenAI...) ?

R. Confidentialité du code source (rien ne sort du réseau), coût nul, et maîtrise du déploiement.
Contrainte assumée : GPU 4GB VRAM, d'où le cap de 1500 caractères sur le skeleton et le choix d'un modèle 6.7b spécialisé code (deepseek-coder).

Q. Comment garantir que le test généré compile et s'exécute ?

R. Prompt structuré (méthode + scénario + schéma Swagger), règles strictes (package, mocks, nommage), puis exécution réelle via Maven et parsing Surefire. L'analyse IA des échecs classe le type d'erreur (COMPILATION_ERROR, etc.) pour fermer la boucle qualité.

Q. Pourquoi TestNG plutôt que JUnit ?

R. TestNG gère mieux le paramétrage, les groupes et l'ordre, et s'intègre au provider Surefire configuré. Tout le code généré et le runner sont alignés sur TestNG pour éviter les conflits de framework.

Q. Pourquoi une base PostgreSQL partagée entre microservices ?

R. Choix pragmatique pour un PFE : pas de duplication ni de synchronisation. Le découplage est applicatif (ms-execution lit des miroirs read-only, ddl-auto=validate) plutôt qu'au niveau base.

Q. Comment décider du verdict d'un test fonctionnel web ?

R. Trois oracles indépendants (HTML5 Validity, erreurs DOM, capture réseau XHR/fetch) fusionnés par une table de décision déterministe. Les cas ambigus passent en HITL (validation humaine) en mode supervisé.

Q. Pourquoi ne pas utiliser le CDP (Chrome DevTools Protocol) ?

R. L'approche JavaScript injecté est portable sur tous les navigateurs et plus simple à maintenir.
L'interception réseau se fait par monkey-patch de XMLHttpRequest et wrap de fetch côté page.

Q. Comment l'authentification est-elle gérée ?

R. JWT en cookies (access/refresh), OAuth GitHub, et un proxy de l'API GitHub côté service Auth. Les appels service-à-service passent par des endpoints /api/internal/ sans contrôle de sécurité.

Q. Comment fonctionnent Pause/Reprendre et les flaky tests ?

R. Pause : un Set concurrent d'IDs pausés, vérifié à chaque itération (awaitIfPaused, poll 500ms).
Flaky : un test marqué flaky=true et échoué est automatiquement rejoué une fois.

Q. Quelle est la limite actuelle et la suite ?

R. Auto-correction sur erreur de compilation (renvoyer code + erreur au LLM), génération multi-scénarios en lot, et vrais tests E2E via Playwright. Voir la section « What's Left To Do » du projet.